Experiment 3

Aim: Write a program to check whether a string include keyword or not.

Theory:

A keyword is a reserved word in a programming language that has a fixed, predefined meaning and cannot be used as an identifier, such as int, if, while, or return. An identifier, on the other hand, is a name chosen by the programmer to represent variables, functions, or other user-defined entities, and must not clash with any keyword. Identifying keywords in source code is one of the basic tasks of a lexical analyzer, since recognizing them correctly is what allows the compiler to distinguish language constructs (like control statements) from ordinary variable names. In practice, this is done by first checking whether a scanned token matches any entry in a fixed table of reserved words; if it does, it's classified as a keyword, and if not, it's treated as an identifier. Programs that count keyword occurrences in a paragraph, classify a single word, or total the number of keywords in a sentence all rely on this same core lookup: breaking the input into individual words (tokens) and comparing each one against the keyword list.

Practice Questions

1. **Write a program to identify and count the occurrences of keywords in a given input string/paragraph.**

Code:

```
#include <iostream>

#include <vector>

#include <string>

#include <map>

using namespace std;


vector<string> keywords = {

   "int","float","double","char","if","else","for","while","do",

   "switch","case","break","continue","return","void","struct",

   "static","const"

};
```

```
bool isKeyword(const string &word) {
    for (const string &k : keywords) {
        if (word == k) return true;
    }
    return false;
}

vector<string> tokenize(const string &text) {
    vector<string> tokens;
    string current = "";
    for (char c : text) {
        if (isalnum(c) || c == '_') {
            current += c;
        } else {
            if (!current.empty()) {
                tokens.push_back(current);
                current = "";
            }
        }
    }
    if (!current.empty()) tokens.push_back(current);
    return tokens;
}

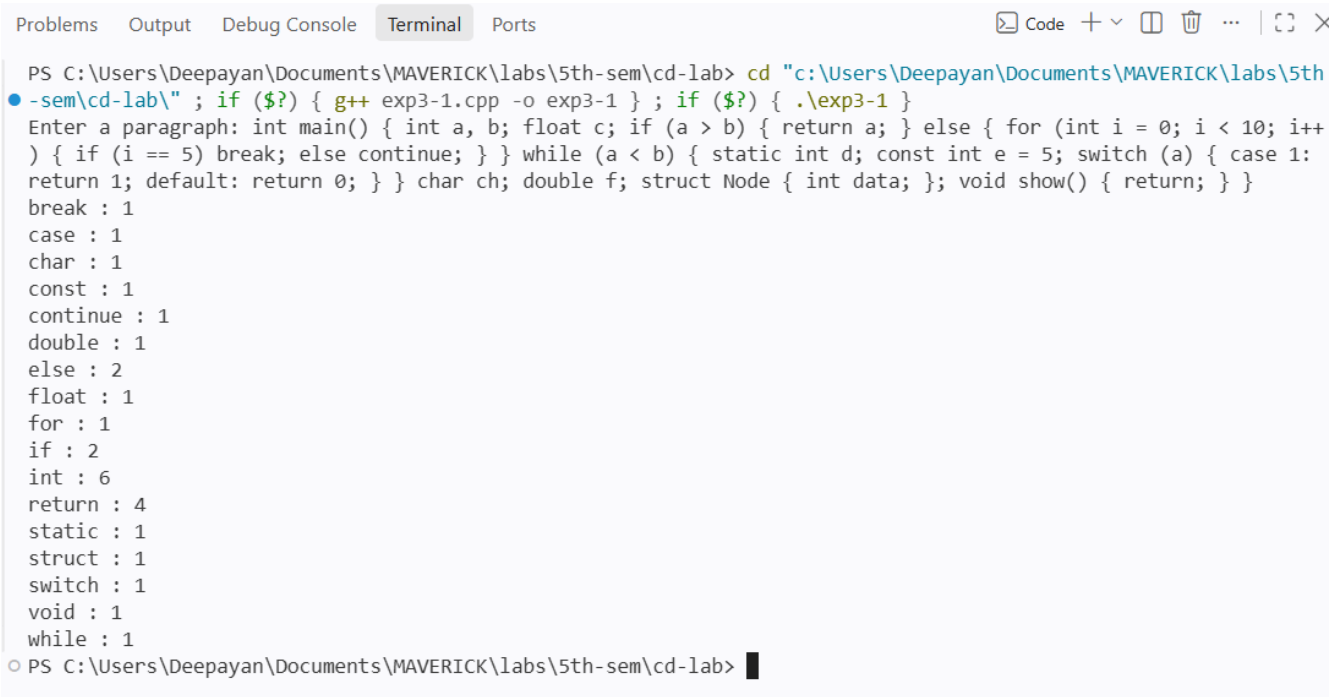
int main() {
    string paragraph;
    cout << "Enter a paragraph: ";
    getline(cin, paragraph);
```

```
    map<string, int> counts;
    for (const string &token : tokenize(paragraph)) {
        if (isKeyword(token)) counts[token]++;
    }

    for (const auto &pair : counts) {
        cout << pair.first << " : " << pair.second << "\n";
    }

    return 0;
}
```

Output:



2. Write a program to check whether a given word is a keyword or an identifier.

Code:

```
#include <iostream>
#include <vector>
#include <string>
```

```
using namespace std;

vector<string> keywords = {
    "int","float","double","char","if","else","for","while","do",
    "switch","case","break","continue","return","void","struct",
    "static","const"
};

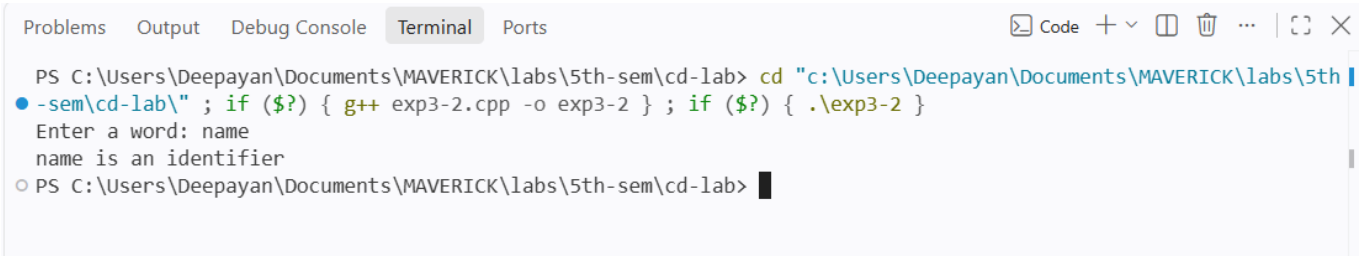
bool isKeyword(const string &word) {
    for (const string &k : keywords) {
        if (word == k) return true;
    }
    return false;
}

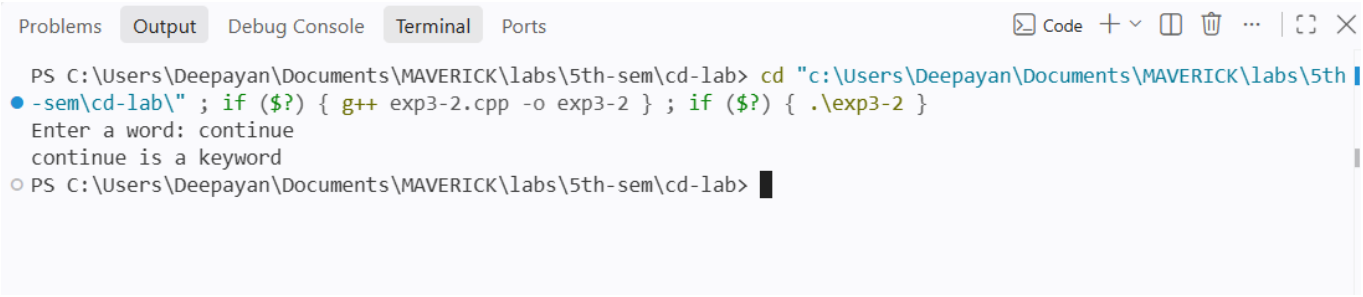
int main() {
    string word;
    cout << "Enter a word: ";
    cin >> word;

    if (isKeyword(word))
        cout << word << " is a keyword\n";
    else
        cout << word << " is an identifier\n";

    return 0;
}
```

Output:





3. Write a program to count the number of keywords in a given sentence.

Code:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

vector<string> keywords = {
    "int","float","double","char","if","else","for","while","do",
    "switch","case","break","continue","return","void","struct",
    "static","const"
};

bool isKeyword(const string &word) {
    for (const string &k : keywords) {
        if (word == k) return true;
    }
    return false;
}
```

```
vector<string> tokenize(const string &text) {
    vector<string> tokens;
    string current = "";
    for (char c : text) {
        if (isalnum(c) || c == '_') {
            current += c;
        } else {
            if (!current.empty()) {
                tokens.push_back(current);
                current = "";
            }
        }
    }
    if (!current.empty()) tokens.push_back(current);
    return tokens;
}

int main() {
    string sentence;
    cout << "Enter a sentence: ";
    getline(cin, sentence);

    int count = 0;
    for (const string &token : tokenize(sentence)) {
        if (isKeyword(token)) count++;
    }
```

```
    cout << "Number of keywords = " << count << "\n";
    return 0;
}
```

Output:

